

The Skateboard: Early Trials

AI-assisted Fortran→GPU porting across four models

Companion to `skateboard.tex` and `architecture.tex`

July 30, 2026

1 What these trials are

We ran the skateboard harness (see `architecture.tex`) on one porting task and four AI models. The task: take one Fortran subroutine that advances the non-linear one-dimensional shallow water equations by a time step, and make it run on an NVIDIA GPU without changing what it computes. Each model’s output is driven through a fixed gate sequence—build, GPU device-execution proof, memory/race sanitizer, correctness on visible data, timing, then correctness on a held-out dataset the model never sees. The agent may edit exactly one file; everything that judges its work is out of its reach.

Two studies are reported. The *model matrix* runs four models once each (up to three repair attempts). The *iteration study* gives the weakest model, a locally-served 14-billion-parameter model, twenty attempts, to see whether persistence substitutes for capability.

2 The constraints the port must satisfy

Every claim below is enforced mechanically by a component the model cannot edit.

The code and the dialect. The target is chapter 4 of the *Modern Fortran* “tsunami” example (Curcic). It is Fortran 2008: `iso_fortran_env` 32-bit reals (`real32`), assumed-shape array arguments, and *standard parallelism* via `do concurrent`. The original kernel is written in whole-array form and calls an array-valued function for the 2nd-order centered difference. The port target is a single subroutine, `step(h,u)`, that updates water height `h` and velocity `u` in place; the momentum update is computed first from the old fields, then the continuity update uses the freshly-updated velocity and the old height. The agent may modify only `src/mod.kernel.f90`.

What the compiler will accept. The build gate is NVIDIA `nvfortran` 25.9 invoked as `-O2 -stdpar=gpu -gpu=cc89,mem:managed -Minfo=accel`. Under this flag a `do concurrent` loop is turned into a GPU kernel. The compiler is strict about what it will offload: it rejected a `block` construct placed inside a `do concurrent` (Sonnet’s first attempt), and it rejected indexing a function result as `diff_centered(u)(i)` together with an undeclared loop variable (qwen’s first attempts). Whole-array assignments that call array-valued functions compile, but run on the host rather than the device. Automatic and allocatable temporaries are accepted. The compiler flags, the exact command, and the environment are baked into the builder image; the agent’s patch can change Fortran source only, never how it is compiled.

How the GPU is used. The method is ISO standard parallelism, not CUDA and not directives: each `do concurrent` loop nest becomes one CUDA kernel, generated by `nvfortran -stdpar=gpu` for compute capability 8.9 (an RTX 4000 Ada). The harness does not trust a successful build to mean the code ran on the device. It sets `NVCOMPILER_ACC_NOTIFY=1` and counts the “launch CUDA kernel” lines the runtime prints; the device-execution gate fails if that count is zero. A directive port that silently falls back to the host is therefore caught, not accepted. NVIDIA `compute-sanitizer` (memcheck, racecheck) must also run clean on the device.

The memory region shared with the GPU. Data is shared through CUDA *managed* (unified) memory, requested by `-gpu=mem:managed`. The height and velocity arrays—at the production size, two million `real32` elements, about 8 MB each—plus the per-step difference and flux temporaries are allocated as managed memory. Managed pages migrate between host and device on demand, at page granularity, and stay resident on the device across successive kernel launches as long as only the GPU touches them. This detail decides performance: a port that keeps the arrays on the device between steps runs fast, whereas a port that also performs a whole-array copy on the host (for example `h_old = h` or `u = u_new`) forces those two-million-element arrays to migrate host↔device every step, and pays for it (Section 3).

What the oracle accepts. Correctness is decided by capture-replay against reference outputs the agent never sees. For each of five “visible” single-step cases the ported kernel is run and its outputs compared, per variable, to the reference. An element passes if any of three metrics is within tolerance—absolute $\leq 10^{-6}$, relative $\leq 10^{-5}$, or units-in-the-last-place ≤ 16 ; a variable passes only if every one of its elements passes. The bands were calibrated from the spread the CPU code already exhibits between `-O2` and `-ffast-math` builds. A separate held-out set of five cases, generated from a different initial condition, is compared only after every other gate passes, and the oracle returns pass/fail for it alone—never a numeric error—so nothing quantitative about the held-out data can leak back to the agent. The reference outputs, the tolerances, and the comparator live in a read-only service on a network the agent cannot reach.

The baseline. Speed is measured end to end on a production-sized run: the domain is 20,000 copies of the 100-point chapter-4 domain laid end to end (two million points), marched 5,000 explicit forward-Euler steps. Because each tile evolves identically under the periodic scheme, the small single-step cases remain an exact correctness reference for the large timing run. Two baselines are measured *before* any model runs: the best CPU configuration of the original code (`nvfortran -O2 -stdpar=multicore`, all cores) and the naive compiler-only GPU attempt (the unmodified whole-array code compiled with `-stdpar=gpu`). In the model-matrix run these were 10.90 s and 7.95 s respectively. Timings are whole-process wall clock and therefore include CUDA start-up, which is the honest end-to-end number.

3 Results

3.1 The model matrix

Four models, one attempt-ladder each (up to three repair attempts on the `do concurrent` strategy). Table 1 gives the outcome. Two Claude models produced accepted ports; the Gemini and local models never produced a numerically correct one. Crucially, no wrong port was ever accepted—the

models that failed did so *at a gate*, with the oracle or the compiler refusing them, not by the harness overlooking an error.

Model	Backend	Outcome	Attempts	Speedup
Claude Opus 4.8	Anthropic API	accepted	1	12.2×
Claude Sonnet 5	Anthropic API	accepted	2	8.9×
Claude Haiku 4.5	Anthropic API	accepted (slow)	1	0.13×
Gemini 2.5 Flash	OpenAI-compat. API	not accepted	3 (build)	—
qwen2.5:14b	local (Ollama)	not accepted	3 (build)	—

Table 1: Model matrix. “Attempts” is the number used before acceptance, or the cap with the stage that failed. Opus is from a separate single-model run under the same setup; the others are one campaign. Speedup is versus the best CPU configuration.

Three findings sit inside that table.

The feedback loop, not raw generation, produced Sonnet’s success. Sonnet’s first attempt failed to build: it placed a `block` construct inside a `do concurrent`, which the GPU offload cannot compile. The structured compiler error was fed back, and the second attempt removed the block and was accepted at 8.9×. The harness turned a failure into a success by returning the right diagnostic, which is the leverage point the design bets on.

Correct is not the same as effective, and the timing gate showed it. Haiku produced a genuinely correct port—it passed the device proof, the sanitizer, and both the visible and held-out comparisons—that nonetheless ran *eight times slower than the CPU* (87.0s versus 10.9s). The cause is the managed-memory effect described above: Haiku’s kernel keeps whole-array host copies, so the two-million-element arrays migrate every step. We record this as a first-class data point rather than hiding it; the point of an honest performance measurement is to surface exactly this.

Weak generation fails safely. Gemini Flash and qwen2.5 never compiled a valid port in three attempts. Their output stopped at the build gate; the oracle never ran on it. The harness’s guarantee is not that every model succeeds, but that a model which cannot do the job is caught rather than trusted.

Figures 1 and 2 show the accepted ports against the two baselines, as speedup and as wall-clock time.

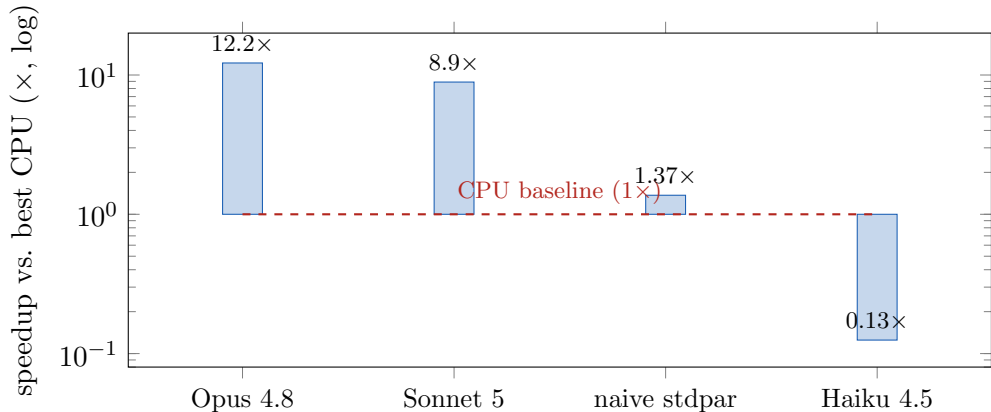


Figure 1: Speedup of each accepted port and of the naive compiler-only GPU attempt, relative to the best CPU build. Above the dashed line is faster than the CPU; Haiku’s correct-but-slow port sits below it.

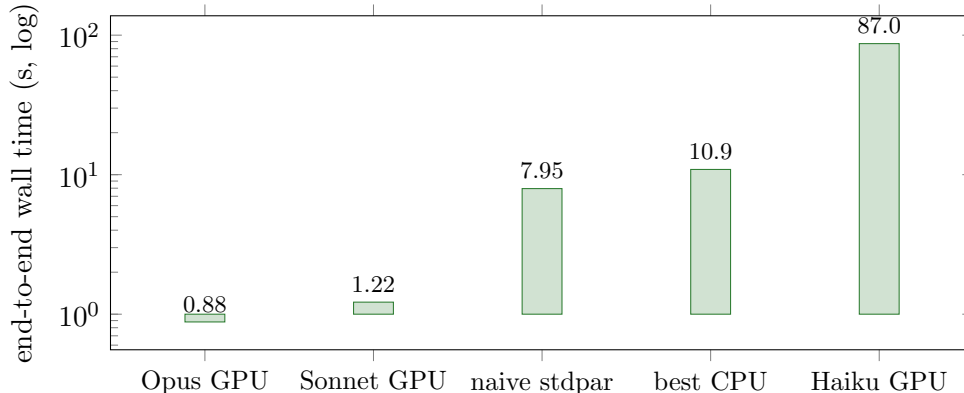


Figure 2: Wall-clock time for the two-million-point, 5000-step run: the two fast accepted GPU ports, the two CPU baselines, and Haiku’s slow GPU port. Whole-process time, including CUDA start-up.

3.2 The iteration study: twenty attempts with a 14B local model

We then gave qwen2.5:14b, served locally through Ollama, twenty attempts on the same task, with the repair loop strengthened so each attempt sees its own previous failing code alongside the compiler or comparator diagnostic. Figure 3 shows what happened. The first three attempts failed to compile. From the fourth attempt onward the model produced code that *compiled and launched GPU kernels*—it crossed the build barrier and passed the device-execution proof. But it never passed the correctness oracle: seventeen consecutive attempts produced plausible, compiling, GPU-executing ports that computed the wrong answer, and the oracle rejected every one. None was accepted.

This is the clearest single demonstration of the design’s premise. Build success is not correctness. A model too weak to get the numerics right can still get past the compiler and onto the device; the only thing standing between “it built and ran on the GPU” and “it is accepted” is the oracle, and here the oracle held the line seventeen times without a single false accept.

Generation time was 10–18s per attempt (the model was already resident, so there was no cold-load penalty this run); the compiling attempts also paid a build and replay cost. The twenty attempts cost roughly five minutes of local generation and no API spend.

4 Reading the results

The four models span the full spectrum the harness is built to sort. Opus and Sonnet cleared every gate and delivered eight- to twelve-fold speedups. Haiku cleared the correctness gates but not the performance bar, and we keep that 0.13× as evidence that “on the GPU and correct” does not imply “fast.” Gemini Flash and qwen could not produce a correct port at all: Gemini stopped at the compiler, and qwen—given twenty tries—got as far as compiling, correct- looking, GPU-executing code that was numerically wrong, and was stopped by the oracle every time.

The through-line is that the outcomes were decided by the harness, not asserted by the model. Every failure landed on a specific gate—the compiler, the device-execution proof, or the correctness oracle—and no wrong or ineffective port was ever accepted. That is the property the skateboard exists to demonstrate, and these early trials demonstrate it across a wide range of model capability, two commercial APIs, and one model running locally on the same machine.

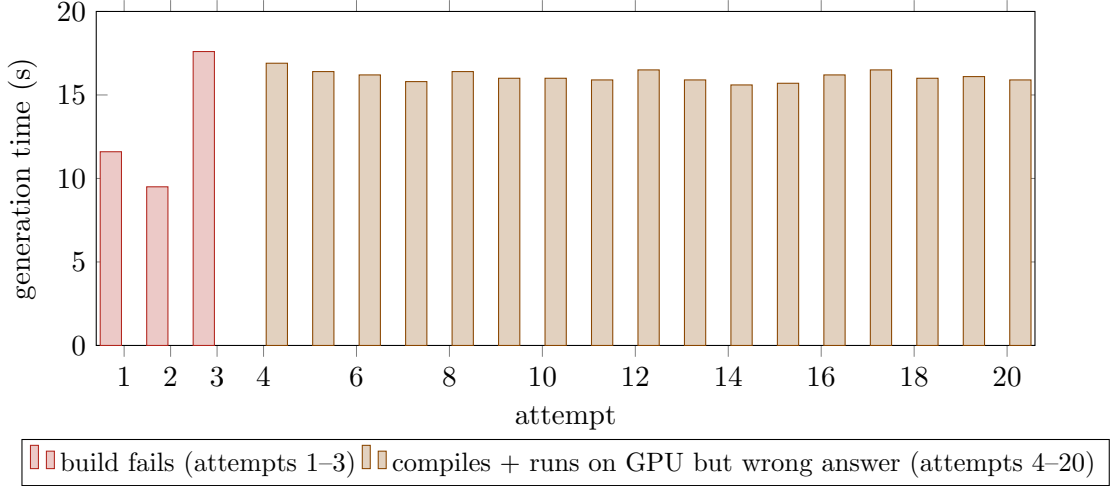


Figure 3: qwen2.5:14b over twenty attempts. Bar height is per-attempt generation time. Red: the port did not compile. Amber: the port compiled and launched GPU kernels but failed the correctness oracle. The build barrier is crossed at attempt 4; the correctness barrier is never crossed.

Caveats. Timings include per-process CUDA start-up. Absolute CPU baselines drift a few percent run to run; speedups, being within-run ratios, do not. Per-attempt wall time is measured directly only for the local model, via the Ollama request log; for the API models we report attempt counts and the measured end-to-end port times, not per-call latency. The correctness tolerances were calibrated from CPU compiler spread and should be re-calibrated against `nvfortran` before any external claim. All runs are on one RTX 4000 Ada; no portability across GPUs or compilers is claimed.